Name- Harshi jain | Roll no. – 21
Branch- B.tech Cse A | CUID- CU240250174

Lab Sheet 10

1. System Architecture

```
campusconnect/
├── backend/
│   ├── server.js                # entry point: HTTP server + Socket.io + Mongo connect │   ├──
src/
│   │   ├── app.js                 # Express app factory (helmet, cors, routes) │   │   ├── config/
(db.js, redis.js)
│   │   ├── models/  (User.js, Event.js, Announcement.js)
│   │   ├── middleware/ (auth.js, rateLimiter.js, validate.js, cache.js)
│   │   ├── controllers/ (authController.js, eventController.js, announcementController.js) │   │
├── routes/  (authRoutes.js, eventRoutes.js, announcementRoutes.js)
│   │   └── sockets/index.js
│   ├── tests/       (auth.test.js, events.test.js, mocks/)
│   ├── benchmark/cache-benchmark.js
│   └── Dockerfile
├── frontend/
│   ├── src/
│   │   ├── app/store.js
│   │   ├── api/axios.js
│   │   ├── features/auth/authSlice.js, features/events/eventsSlice.js
│   │   ├── components/ (PrivateRoute.jsx, RoleRoute.jsx)
│   │   ├── pages/ (Login.jsx, Register.jsx, Dashboard.jsx, EventList.jsx) │   │
├── socket.js, App.jsx
│   │   └── tests/ (Login.test.jsx, EventList.test.jsx)
│   └── Dockerfile, nginx.conf
├── docker-compose.yml
└── README.md
```

2. Task 1 — Authentication & RBAC

src/models/User.js

```
const mongoose = require('mongoose');


const UserSchema = new mongoose.Schema(
  {     name: { type: String, required: true, trim: true },     email: { type: String, required: true, unique:
true, lowercase: true, trim: true },     passwordHash: { type: String, required: true },
    role: { type: String, enum: ['ADMIN', 'STUDENT'], default: 'STUDENT' },
    refreshTokenVersion: { type: Number, default: 0 }, // bump to invalidate all refresh tokens
  },
  { timestamps: true }
```

```
);

  module.exports = mongoose.models.User || mongoose.model('User', UserSchema);
```

src/utils/jwt.js

```
const jwt = require('jsonwebtoken');                                                                const

const ACCESS_SECRET = process.env.JWT_ACCESS_SECRET || 'dev_access_secret_change_me';
REFRESH_SECRET = process.env.JWT_REFRESH_SECRET || 'dev_refresh_secret_change_me';

function signAccessToken(user) {   return jwt.sign(
   { sub: user._id.toString(), role: user.role, email: user.email },    ACCESS_SECRET,
{ expiresIn: '15m' }
 );
}

function signRefreshToken(user) {   return jwt.sign(
   { sub: user._id.toString(), v: user.refreshTokenVersion || 0 },    REFRESH_SECRET,
   { expiresIn: '7d' }
 );
}

function verifyAccessToken(token) {    return jwt.verify(token,
ACCESS_SECRET);
}

function verifyRefreshToken(token) {   return jwt.verify(token,
REFRESH_SECRET);
}

module.exports = { signAccessToken, signRefreshToken, verifyAccessToken, verifyRefreshToken,
ACCESS_SECRET, REFRESH_SECRET };
```

src/middleware/auth.js — authenticate & authorize(role)

```
const { verifyAccessToken } = require('../utils/jwt');


// Verifies the Bearer access token and attaches req.user = { sub, role, email } function
authenticate(req, res, next) {   const header = req.headers.authorization || '';
const token = header.startsWith('Bearer ') ? header.slice(7) : null;


  if (!token) {
    return res.status(401).json({ error: 'Missing access token' });   }     try {
const payload = verifyAccessToken(token);     req.user = payload;     return
next();   } catch (err) {
    return res.status(401).json({ error: 'Invalid or expired access token' });   }
}
// Usage: authorize('ADMIN') or authorize('ADMIN', 'STUDENT')
function authorize(...allowedRoles) {   return (req, res, next) => {     if
(!req.user) {       return res.status(401).json({ error: 'Not authenticated'
});
    }
    if (!allowedRoles.includes(req.user.role)) {       return res.status(403).json({ error: 'Forbidden:
insufficient role' });
    }     return next();
  };
}
```

```
module.exports = { authenticate, authorize };
```

src/controllers/authController.js

```
const bcrypt = require('bcryptjs'); const User
= require('../models/User');
const { signAccessToken, signRefreshToken, verifyRefreshToken } =
require('../utils/jwt');

const SALT_ROUNDS = 10;

const REFRESH_COOKIE_OPTS = {
httpOnly: true,
  secure: process.env.NODE_ENV === 'production',   sameSite:
'strict',
  maxAge: 7 * 24 * 60 * 60 * 1000, // 7 days
path: '/api/auth', };

async function register(req, res) {    const { name, email,
password, role } = req.body;

  const existing = await User.findOne({ email });     if
(existing) {
    return res.status(409).json({ error: 'Email already registered' });   }

  const passwordHash = await bcrypt.hash(password,
SALT_ROUNDS);   const user = await User.create({     name,
email,     passwordHash,
    role: role === 'ADMIN' ? 'ADMIN' : 'STUDENT',   });

  const accessToken = signAccessToken(user);   const
refreshToken = signRefreshToken(user);
  res.cookie('refreshToken', refreshToken, REFRESH_COOKIE_OPTS);

  return res.status(201).json({
    user: { id: user._id, name: user.name, email: user.email, role: user.role },
accessToken,
  });
}

async function login(req, res) {     const {
email, password } = req.body;

  const user = await User.findOne({ email });   if
(!user) {
    return res.status(401).json({ error: 'Invalid email or password' });   }

  const valid = await bcrypt.compare(password, user.passwordHash);   if
(!valid) {     return res.status(401).json({ error: 'Invalid email or password' });
}

  const accessToken = signAccessToken(user);   const refreshToken =
signRefreshToken(user);   res.cookie('refreshToken', refreshToken,
REFRESH_COOKIE_OPTS);
```

```
  return res.status(200).json({
    user: { id: user._id, name: user.name, email: user.email, role: user.role },     accessToken,
  });
}

async function refresh(req, res) {
  const token = req.cookies && req.cookies.refreshToken;   if
(!token) {
    return res.status(401).json({ error: 'Missing refresh token' });
  }

  let payload;   try {
    payload = verifyRefreshToken(token);
  } catch (err) {
    return res.status(401).json({ error: 'Invalid or expired refresh token' });   }

  const user = await User.findById(payload.sub);   if (!user || (user.refreshTokenVersion ||
0) !== payload.v) {     return res.status(401).json({ error: 'Refresh token revoked' });   }

  const accessToken = signAccessToken(user);   return res.status(200).json({ accessToken
});
}

function logout(req, res) {
  res.clearCookie('refreshToken', { path: '/api/auth' });   return res.status(204).send();  }

module.exports = { register, login, refresh, logout };
```

src/routes/authRoutes.js

```
const express = require('express'); const { z
} = require('zod');
const validate = require('../middleware/validate');
const { loginLimiter } = require('../middleware/rateLimiter');
const { register, login, refresh, logout } = require('../controllers/authController');

const router = express.Router();

const registerSchema = z.object({   name:     z.string().min(2).max(100),
email: z.string().email(),    password: z.string().min(8, 'Password must be at
least 8 characters'),   role: z.enum(['ADMIN', 'STUDENT']).optional(), });

const loginSchema = z.object({   email:
z.string().email(),    password: z.string().min(1,
'Password is required'), });
```

```
router.post('/register', validate({ body: registerSchema }), register); router.post('/login',
loginLimiter, validate({ body: loginSchema }), login); router.post('/refresh', refresh);
router.post('/logout', logout);


module.exports = router;
```

3. Task 2 — Real-Time Notifications

src/sockets/index.js (backend)

```
const { Server } = require('socket.io'); const { verifyAccessToken } = require('../utils/jwt');

function initSocket(httpServer, corsOrigin) {
```

```
  const io = new Server(httpServer, {
    cors: { origin: corsOrigin, credentials: true },
    // Helps clients recover missed events across brief disconnects
connectionStateRecovery: { maxDisconnectionDuration: 2 * 60 * 1000 },   });


  // Authenticate every socket connection using the same JWT access token
// used for REST calls, sent via the `auth` payload during the handshake.
io.use((socket, next) => {     try {       const token = socket.handshake.auth
&& socket.handshake.auth.token;       if (!token) return next(new
Error('Unauthorized: missing token'));


      const payload = verifyAccessToken(token);
socket.user = payload; // { sub, role, email }       return
next();     } catch (err) {
      return next(new Error('Unauthorized: invalid or expired token'));
    }
  });


  io.on('connection', (socket) => {
    // Put every socket in a room named after its role so we can broadcast     // "new-
announcement" to all STUDENT sockets in one call.     socket.join(socket.user.role);
socket.join(`user:${socket.user.sub}`);


    console.log(`[socket] connected: ${socket.user.email} (${socket.user.role})`);


    socket.on('disconnect', (reason) => {
      console.log(`[socket] disconnected: ${socket.user.email} (${reason})`);     });


    // Client-side reconnection is handled automatically by socket.io-client     //
(exponential backoff by default); this event just lets us log it.
socket.on('reconnect_attempt', (attempt) => {
      console.log(`[socket] reconnect attempt #${attempt} for ${socket.user.email}`);     });
  });

return
io;  }
module.exports = initSocket;
```

src/controllers/announcementController.js — emits the event

```
async function createAnnouncement(req, res) {   const { title, body } = req.body;   const
announcement = await Announcement.create({ title, body, createdBy: req.user.sub });

  // req.io is attached in server.js so controllers can emit without a circular import    const io =
req.app.get('io');   if (io) {
    io.to('STUDENT').emit('new-announcement', {
id: announcement._id,       title: announcement.title,
body: announcement.body,
      createdAt: announcement.createdAt,     });
  }
  return res.status(201).json(announcement);
}

async function listAnnouncements(req, res) {
  const announcements = await Announcement.find().sort({ createdAt: -1 }).limit(50).lean();
return res.status(200).json(announcements);
}

module.exports = { createAnnouncement, listAnnouncements };
```

frontend/src/socket.js — client connection + live badge

```
const Announcement = require('../models/Announcement');
```

```
import { io } from 'socket.io-client'; import {
store } from './app/store';
import { announcementReceived } from './features/events/eventsSlice';


let socket = null;


export function connectSocket() {
  const token = store.getState().auth.accessToken;   if (!token)
return null;


  socket = io(import.meta.env.VITE_SOCKET_URL || 'http://localhost:5000', {
auth: { token },        reconnection: true,          reconnectionAttempts: Infinity,
reconnectionDelay: 1000,     reconnectionDelayMax: 5000,   });


  socket.on('connect', () => console.log('[socket] connected', socket.id));


  // Gracefully handle drops: socket.io-client auto-reconnects; we just log   // and re-authenticate
automatically since `auth` is re-sent on reconnect.   socket.on('disconnect', (reason) =>
console.log('[socket] disconnected:', reason));   socket.on('connect_error', (err) =>
console.log('[socket] connect_error:', err.message));


  socket.on('new-announcement', (announcement) => {
store.dispatch(announcementReceived());
    // A toast library (e.g. react-hot-toast) would be wired in here:
// toast(`New announcement: ${announcement.title}`);
   console.log('[socket] new-announcement received:', announcement.title);   });
return socket;  }


export function disconnectSocket() {   if
(socket) socket.disconnect();
}
```

4. Task 3 — Redis Caching Layer

src/middleware/cache.js

```
const redisClient = require('../config/redis');

const EVENTS_CACHE_KEY_PREFIX = 'events:list:'; const EVENTS_TTL_SECONDS = 60;

// Caches full JSON responses for GET /api/events, keyed by the query string   // (so
/api/events?page=2&search=fest   caches   separately   from   page=1).   function
cacheEvents(req, res, next) {
  const cacheKey = EVENTS_CACHE_KEY_PREFIX + JSON.stringify(req.query);
    redisClient
.get(cacheKey)
.then((cached) => {       if
(cached) {
        res.set('X-Cache', 'HIT');
        return res.status(200).json(JSON.parse(cached));       }


      res.set('X-Cache', 'MISS');
      // Monkey-patch res.json so we cache whatever the controller sends      const
originalJson = res.json.bind(res);       res.json = (body) => {
        redisClient.set(cacheKey, JSON.stringify(body), 'EX', EVENTS  TTL  SECONDS).catch((err) => {
console.error('[cache] failed to SET', err.message);
        });
        return originalJson(body);
      };
      return next();
    })
    .catch((err) => {
      console.error('[cache] redis GET failed, falling back to DB:', err.message);       return next();
    });
}
// Invalidate every cached events list (any page/search/filter combination) async function
invalidateEventsCache() {
  const keys = await
redisClient.keys(`${EVENTS_CACHE_KEY_PREFIX}*`);   if (keys.length)
{
    await redisClient.del(...keys);   }
}


module.exports = { cacheEvents, invalidateEventsCache, EVENTS_TTL_SECONDS };
```

Terminal — redis-cli ping

```
$ redis-cli ping PONG
```

Terminal — node benchmark/cache-benchmark.js

```
[redis] connected


=== CampusConnect Redis Cache Benchmark (Task 3) ===
Requests per round: 150
Cache hits in cached round: 150/150


-- Uncached (DB hit every request) -- avg: 1.205 ms  p95: 2.020 ms  min: 0.825
ms  max: 4.111 ms
   -- Cached (Redis, 60s TTL) -- avg: 1.061 ms  p95: 3.338 ms  min: 0.646 ms
max:
5.484 ms


Improvement: 12.0% faster average response time with caching enabled.
```

5. Task 4 — Frontend with State Management

src/app/store.js

```
import { configureStore } from '@reduxjs/toolkit'; import
authReducer from '../features/auth/authSlice'; import
eventsReducer from '../features/events/eventsSlice';


export const store = configureStore({   reducer:
{     auth: authReducer,
events: eventsReducer,
  },
});
```

```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit'; import api from
'../../api/axios';


export const login = createAsyncThunk('auth/login', async ({ email, password }, { rejectWithValue
}) => {   try {
    const { data } = await api.post('/auth/login', { email, password });     return
data;   } catch (err) {
    return rejectWithValue(err.response?.data?.error || 'Login failed');
  }
});


export const register = createAsyncThunk('auth/register', async (payload, { rejectWithValue
}) => {   try {
    const { data } = await api.post('/auth/register', payload);     return
data;   } catch (err) {
    return rejectWithValue(err.response?.data?.error || 'Registration failed');
```

```
  }
```

src/api/axios.js — in-memory token + auto-refresh interceptor

```
import axios from 'axios';  import
{ store } from '../app/store';
import { setAccessToken, clearAuth } from '../features/auth/authSlice';


// Access token lives only in Redux (in-memory) — never localStorage — so it
// disappears on tab close/refresh, which is why /api/auth/refresh (backed by // the
httpOnly refresh cookie) exists to silently re-issue it. const api = axios.create({
baseURL:      import.meta.env.VITE_API_URL      ||      'http://localhost:5000/api',
withCredentials: true, // sends the httpOnly refresh cookie });


api.interceptors.request.use((config) => {   const token
= store.getState().auth.accessToken;   if (token) {
config.headers.Authorization = `Bearer ${token}`;
  }
  return config; });


let refreshing = null;


api.interceptors.response.use(
  (res) => res,   async
(error) => {
    const original = error.config;
    if (error.response && error.response.status === 401 && !original._retry) {
original._retry = true;       try {         refreshing = refreshing || api.post('/auth/refresh');
const { data } = await refreshing;         refreshing = null;
store.dispatch(setAccessToken(data.accessToken));
       original.headers.Authorization = `Bearer ${data.accessToken}`;         return
api(original);       } catch (refreshErr) {         refreshing = null;
store.dispatch(clearAuth());
      }     }     return
Promise.reject(error);
  } );


export default api;
```

src/features/auth/authSlice.js

```
});


const authSlice = createSlice({   name:
'auth',
  initialState: { user: null, accessToken: null, status: 'idle', error: null },   reducers: {
setAccessToken(state, action) {       state.accessToken = action.payload;
    },     clearAuth(state) {
state.user = null;       state.accessToken
= null;
    },   },
  extraReducers: (builder) => {     builder
.addCase(login.pending, (state) => {
state.status = 'loading';         state.error = null;
})
      .addCase(login.fulfilled, (state, action) => {         state.status
= 'succeeded';         state.user =
action.payload.user;
        state.accessToken = action.payload.accessToken;       })
      .addCase(login.rejected, (state, action) => {
state.status = 'failed';         state.error = action.payload;
})
      .addCase(register.fulfilled, (state, action) => {         state.user =
action.payload.user;
        state.accessToken = action.payload.accessToken;       });
}, });


export const { setAccessToken, clearAuth } = authSlice.actions; export default
authSlice.reducer;
```

src/features/events/eventsSlice.js

```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit'; import api from
'../../api/axios';


export const fetchEvents = createAsyncThunk('events/fetch', async ({ page = 1, search = '' } = {}) => {    const {
data } = await api.get('/events', { params: { page, limit: 10, search } });   return data;  });


export const rsvpToEvent = createAsyncThunk('events/rsvp', async (eventId) => {   const { data }
= await api.post(`/events/${eventId}/rsvp`);   return data; });


const eventsSlice = createSlice({   name:
'events',   initialState: {     items: [],     page: 1,
totalPages: 1,     search: '',     status:
'idle',
    unreadAnnouncements: 0, // live badge counter, bumped by the socket.js listener
  },   reducers: {     setSearch(state, action) {       state.search
= action.payload;
    },     announcementReceived(state) {       state.unreadAnnouncements
+= 1;
    },     clearAnnouncementBadge(state) {       state.unreadAnnouncements = 0;
    },   },
  extraReducers: (builder) => {     builder
      .addCase(fetchEvents.pending, (state) => {         state.status = 'loading';
      })
      .addCase(fetchEvents.fulfilled, (state, action) => {         state.status =
'succeeded';         state.items = action.payload.items;         state.page =
action.payload.page;
```

```
        state.totalPages = action.payload.totalPages;       })
.addCase(rsvpToEvent.fulfilled, (state, action) => {
        const idx = state.items.findIndex((e) => e._id === action.payload._id);         if (idx !== -
1) state.items[idx] = action.payload;       });    }, });


export const { setSearch, announcementReceived, clearAnnouncementBadge } = eventsSlice.actions; export
default eventsSlice.reducer;
```

src/components/PrivateRoute.jsx & RoleRoute.jsx

```
import { useSelector } from 'react-redux'; import {
Navigate, Outlet } from 'react-router-dom';


export    default function PrivateRoute() {                const accessToken =
useSelector((state) => state.auth.accessToken);          return accessToken ?
<Outlet /> : <Navigate to="/login" replace />;
}


import { useSelector } from 'react-redux'; import {
Navigate, Outlet } from 'react-router-dom';


export default function RoleRoute({ allowedRoles }) {   const
user = useSelector((state) => state.auth.user);   if (!user) return
<Navigate to="/login" replace />;
  return allowedRoles.includes(user.role) ? <Outlet /> : <Navigate to="/dashboard" replace />;
}
```

src/pages/EventList.jsx — pagination, search, RSVP

```
import { useEffect } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { fetchEvents, rsvpToEvent, setSearch } from '../features/events/eventsSlice';


export default function EventList() {   const
dispatch = useDispatch();
  const { items, page, totalPages, search, status } = useSelector((state) => state.events);   const user =
useSelector((state) => state.auth.user);


  useEffect(() => {
    dispatch(fetchEvents({ page: 1, search }));
    // eslint-disable-next-line react-hooks/exhaustive-deps   },
[search]);
return (
   <div>
     <h2>Campus Events</h2>
     <input        aria-label="search-events"        placeholder="Search
by title..."
value={search}
       onChange={(e) => dispatch(setSearch(e.target.value))}       />


     {status === 'loading' && <p>Loading events...</p>}
```

```
      <ul data-testid="event-list">
        {items.map((event) => (
<li key={event._id}>
           <strong>{event.title}</strong> — {new Date(event.date).toLocaleDateString()}
{user?.role === 'STUDENT' && (
             <button onClick={() => dispatch(rsvpToEvent(event._id))}>RSVP</button>
           )}
          </li>
        ))}
      </ul>


      <div>
        <button disabled={page <= 1} onClick={() => dispatch(fetchEvents({ page: page - 1, search }))}>          Prev
        </button>
        <span>
          Page {page} of {totalPages}         </span>
        <button disabled={page >= totalPages} onClick={() => dispatch(fetchEvents({ page: page + 1, search }))}>
          Next
        </button>
      </div>
    </div>
  );
}
```

6. Task 5 — Security Hardening

src/app.js — helmet, CORS, route wiring, central error handler

```
helmet = require('helmet');
require('cookie-parser');

const authRoutes = require('./routes/authRoutes'); const eventRoutes =
require('./routes/eventRoutes'); const announcementRoutes =
require('./routes/announcementRoutes');

function createApp() {   const
app = express();

  // --- Security hardening (Task 5) ---
app.use(helmet());   app.use(     cors({
      origin: process.env.FRONTEND_ORIGIN || 'http://localhost:5173',           credentials:
true,
    })
  );

  app.use(express.json());   app.use(cookieParser());

  app.get('/api/health', (req, res) => res.json({ status: 'ok' }));

  app.use('/api/auth', authRoutes);              app.use('/api/events',
eventRoutes);
  app.use('/api/announcements', announcementRoutes);
const express = require('express'); const const cors = require('cors');

const cookieParser =

  // Central error handler — keeps stack traces out of API responses
app.use((err, req, res, next) => {     console.error(err);
    res.status(err.status || 500).json({ error: err.message || 'Internal server error' });   });
return app;  }

module.exports = createApp;
```

src/middleware/rateLimiter.js

```
const rateLimit = require('express-rate-limit');

// Max 5 login attempts per 15 minutes per IP const
loginLimiter = rateLimit({   windowMs: 15
* 60 * 1000,   max: 5,    standardHeaders:
true,   legacyHeaders: false,
  message: { error: 'Too many login attempts. Please try again after 15 minutes.' }, });

module.exports = { loginLimiter };
```

src/middleware/validate.js — generic zod validator

```
// Generic zod validator: validate({ body: schema,
query: schema, params: schema }) function
validate(schemas) {   return (req, res, next) => {
try {
      if (schemas.body) req.body =
schemas.body.parse(req.body);       if
(schemas.query) req.query =
schemas.query.parse(req.query);
      if (schemas.params) req.params = schemas.params.parse(req.params);
return next();     } catch (err) {       return res.status(400).json({         error:
'Validation failed',         details: err.errors ? err.errors.map((e) => ({ path:
e.path.join('.'), message: e.message })) : err.message,
      });
    }

}
;
}

module.exports = validate;
```

7. Task 6 — Testing

tests/auth.test.js

```
const request = require('supertest');


jest.mock('../src/models/User', () => require('./mocks/inMemoryUser'));
jest.mock('../src/config/redis', () => new (require('ioredis-mock'))());
jest.mock('../src/middleware/cache', () => ({  cacheEvents: (req, res, next) => next(),
invalidateEventsCache: async () => {}, }));


const createApp = require('../src/app'); const User = require('../src/models/User');


const app = createApp();


beforeEach(() => {
  User.__reset();
});


describe('POST /api/auth/register', () => {
  it('registers a new user and returns an access token', async () => {  const res = await request(app).post('/api/auth/register').send({  name: 'Alice Student',  email: 'alice@campus.edu',  password: 'password123',  role: 'STUDENT',  });


    expect(res.status).toBe(201);
expect(res.body).toHaveProperty('accessToken');
expect(res.body.user.email).toBe('alice@campus.edu');
expect(res.body.user.role).toBe('STUDENT');  // password / hash must never be exposed
expect(res.body.user.passwordHash).toBeUndefined();  });


  it('rejects a weak password via zod validation (400)', async () => {  const res = await request(app).post('/api/auth/register').send({  name: 'Bob',  email: 'bob@campus.edu',
password: '123',
    });
    expect(res.status).toBe(400);
}); });


describe('POST /api/auth/login', () => {
beforeEach(async () => {
    await request(app).post('/api/auth/register').send({
name: 'Carol Admin',  email: 'carol@campus.edu',
password: 'supersecret1',  role: 'ADMIN',
    });
  });


  it('logs in with correct credentials', async () => {
    const res = await request(app).post('/api/auth/login').send({  email: 'carol@campus.edu',  password: 'supersecret1',
    });
    expect(res.status).toBe(200);
    expect(res.body).toHaveProperty('accessToken');  });
```

```
  it('fails login with wrong password (401)', async () => {     const res = await
request(app).post('/api/auth/login').send({       email:
'carol@campus.edu',
      password: 'totally-wrong-password',
    });
    expect(res.status).toBe(401);
    expect(res.body.error).toMatch(/invalid email or password/i);   }); });


describe('Protected routes', () => {
  it('rejects requests to /api/events with no token (401)', async () => {     const res =
await request(app).get('/api/events');     expect(res.status).toBe(401);   });


  it('rejects a STUDENT creating an event via an admin-only route (403)', async () => {     const
reg = await request(app).post('/api/auth/register').send({       name: 'Dave Student',       email:
'dave@campus.edu',       password: 'password123',       role: 'STUDENT',
    });
    const token = reg.body.accessToken;


    const res = await request(app)
      .post('/api/events')
      .set('Authorization', `Bearer ${token}`)
      .send({ title: 'Tech Fest', date: '2026-10-01' });


    expect(res.status).toBe(403);
  });
});
```

tests/events.test.js

```
const request = require('supertest');

jest.mock('../src/models/User', () => require('./mocks/inMemoryUser'));
jest.mock('../src/models/Event', () => require('./mocks/inMemoryEvent'));
jest.mock('../src/config/redis', () => new (require('ioredis-mock'))());
jest.mock('../src/middleware/cache', () => ({  cacheEvents: (req, res, next) =>
next(),  invalidateEventsCache: async () => {}, }));

const createApp = require('../src/app'); const User =
require('../src/models/User');
const Event = require('../src/models/Event');


const app = createApp();


async function registerAdmin() {
  const res = await request(app).post('/api/auth/register').send({     name:
'Admin User',     email: 'admin@campus.edu',     password:
'adminpass123',     role: 'ADMIN',
  });
  return res.body.accessToken;
}
```

```
beforeEach(() => {   User.__reset();
  Event.__reset();
});

describe('POST /api/events', () => {
  it('lets an ADMIN create an event successfully (201)', async () => {     const token
= await registerAdmin();

    const res = await request(app)
      .post('/api/events')
      .set('Authorization', `Bearer ${token}`)
      .send({ title: 'Annual Tech Fest', description: 'Coding & robotics', date: '2026-11-05', venue: 'Main Auditorium' });

    expect(res.status).toBe(201);
    expect(res.body.title).toBe('Annual Tech Fest');     expect(res.body.createdBy).toBeDefined();
  }); });

describe('GET /api/events', () => {
  it('supports pagination and search-by-title', async () => {     const token
= await registerAdmin();

    await request(app).post('/api/events').set('Authorization', `Bearer ${token}`).send({ title: 'Tech Fest', date: '2026-11-05'
});
    await request(app).post('/api/events').set('Authorization', `Bearer ${token}`).send({ title: 'Cultural Night', date: '2026-
11-10' });
    await request(app).post('/api/events').set('Authorization', `Bearer ${token}`).send({ title: 'Tech Talk
Series', date: '2026-11-15' });

    const  res        =        await      request(app).get('/api/events?search=Tech&page=1&limit=10').set('Authorization',
           `Bearer ${token}`);

    expect(res.status).toBe(200);     expect(res.body.total).toBe(2);
expect(res.body.items.every((e) => e.title.includes('Tech'))).toBe(true);   }); });
```

Terminal — npx jest --runInBand --verbose

```
PASS
tests/auth.test.js
POST
/api/auth/register
✓             registers a new user and returns an access
token (153 ms)
✓             rejects a weak password via zod
validation (400) (5 ms)
  POST /api/auth/login
✓             logs in with correct credentials (152 ms)
✓ fails login with wrong password (401) (160 ms)
Protected routes
```

```
✓ rejects requests to /api/events with no token
   (401) (2 ms)
✓ rejects a STUDENT creating an event via an
   admin-only route (403) (84 ms)


PASS tests/events.test.js
 POST /api/events ✓ lets an
ADMIN create an event
successfully (201) (98 ms)
 GET /api/events
✓ supports pagination and search-by-title (83
   ms)


Test Suites: 2 passed, 2 total
Tests:       8 passed, 8 total
Snapshots:   0 total
Time:        1.526 s, estimated 2 s Ran all test
suites.
```

7.2 Frontend — Vitest + React Testing Library (3 tests) **src/tests/Login.test.jsx**

```
import { render, screen, fireEvent, waitFor } from '@testing-library/react'; import { Provider } from
'react-redux';
import { configureStore } from '@reduxjs/toolkit'; import {
describe, it, expect, vi } from 'vitest'; import authReducer from
'../features/auth/authSlice'; import eventsReducer from
'../features/events/eventsSlice';


vi.mock('../api/axios', () => ({   default: {
    post: vi.fn(() => Promise.reject({ response: { data: { error: 'Invalid email or password' } } })),
  },
}));


vi.mock('../socket', () => ({ connectSocket: vi.fn() }));


import Login from '../pages/Login'; import {
MemoryRouter } from 'react-router-dom';


function renderWithStore(ui) {
  const store = configureStore({ reducer: { auth: authReducer, events: eventsReducer } });   return render(
    <Provider store={store}>
      <MemoryRouter>{ui}</MemoryRouter>
    </Provider>
  );
}


describe('Login form', () => {
  it('renders email and password fields', () => {     renderWithStore(<Login />);
expect(screen.getByLabelText(/email/i)).toBeInTheDocument();
expect(screen.getByLabelText(/password/i)).toBeInTheDocument();   });


  it('shows an error message when login fails', async () => {     renderWithStore(<Login />);
fireEvent.change(screen.getByLabelText(/email/i), { target: { value: 'wrong@campus.edu' } });
fireEvent.change(screen.getByLabelText(/password/i), { target: { value: 'badpassword' } });
fireEvent.click(screen.getByRole('button', { name: /login/i }));


    await waitFor(() => {
      expect(screen.getByRole('alert')).toHaveTextContent(/invalid email or password/i);     });
  });
});
```

src/tests/EventList.test.jsx

```
import { render, screen } from '@testing-library/react'; import { Provider }
from 'react-redux';
import { configureStore } from '@reduxjs/toolkit'; import { describe,
it, expect, vi } from 'vitest'; import eventsReducer from
'../features/events/eventsSlice'; import authReducer from
'../features/auth/authSlice';


vi.mock('../api/axios', () => ({   default: { get: vi.fn(() => Promise.resolve({ data: { items: [], page: 1, totalPages: 1,
total: 0 } })) }, }));


import EventList from '../pages/EventList';


describe('EventList', () => {
  it('renders events already present in the Redux store', () => {     const preloadedState
= {
      auth: { user: { role: 'STUDENT' }, accessToken: 'fake-token', status: 'idle', error: null },       events: {         items:
[
          { _id: '1', title: 'Tech Fest', date: '2026-11-05' },
          { _id: '2', title: 'Cultural Night', date: '2026-11-10' },
        ],         page: 1,
totalPages: 1,         search: '',
status: 'succeeded',
unreadAnnouncements: 0,
      },     };
    const store = configureStore({ reducer: { auth: authReducer, events: eventsReducer }, preloadedState });


    render(
      <Provider store={store}>
        <EventList />
      </Provider>
    );


    expect(screen.getByText(/Tech Fest/)).toBeInTheDocument();     expect(screen.getByText(/Cultural
Night/)).toBeInTheDocument();
    expect(screen.getAllByRole('button', { name: /rsvp/i })).toHaveLength(2);   }); });
```

Terminal — npx vitest run

```
 RUN  v2.1.9 /home/claude/campusconnect/frontend


✓ src/tests/Login.test.jsx (2 tests) 107ms  ✓
src/tests/EventList.test.jsx (1 test) 102ms


 Test Files  2 passed (2)
      Tests  3 passed (3)
   Duration  1.94s
```

backend/Dockerfile

```
FROM node:20-alpine


WORKDIR /app
```

```
COPY package*.json ./
RUN npm ci --omit=dev
```

8. Task 7 — Containerization & Deployment

```
COPY . .

ENV NODE_ENV=production  EXPOSE 5000

CMD ["node", "server.js"]
```

frontend/Dockerfile (multi-stage: build → Nginx)

```
# --- Build stage --- FROM node:20-alpine AS
build

WORKDIR /app
COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

# --- Serve stage ---
FROM nginx:1.27-alpine

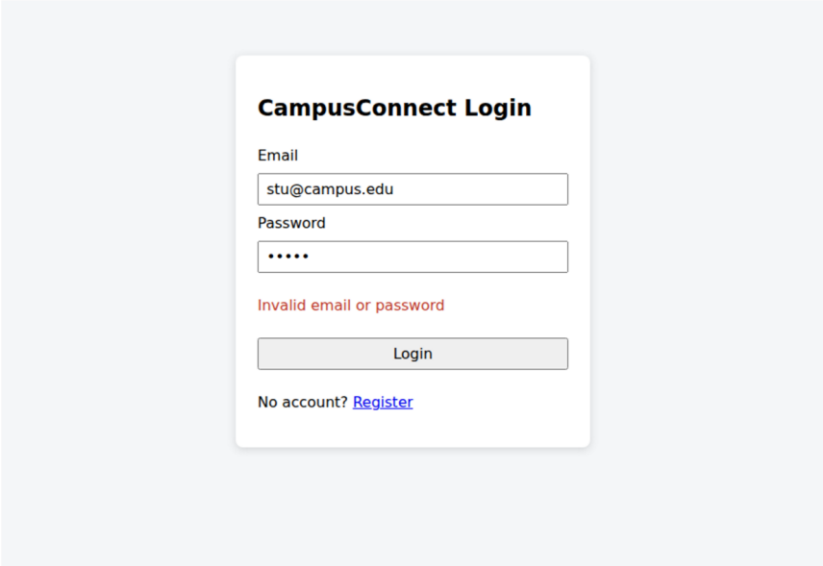
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf

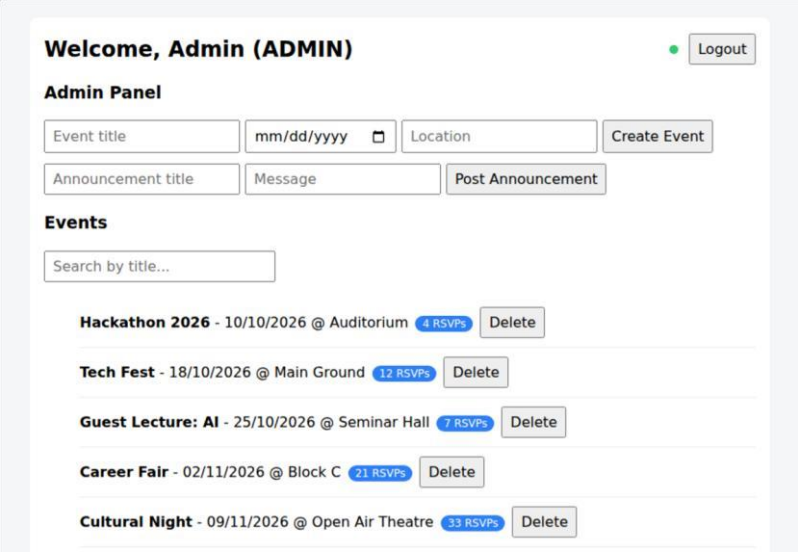
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

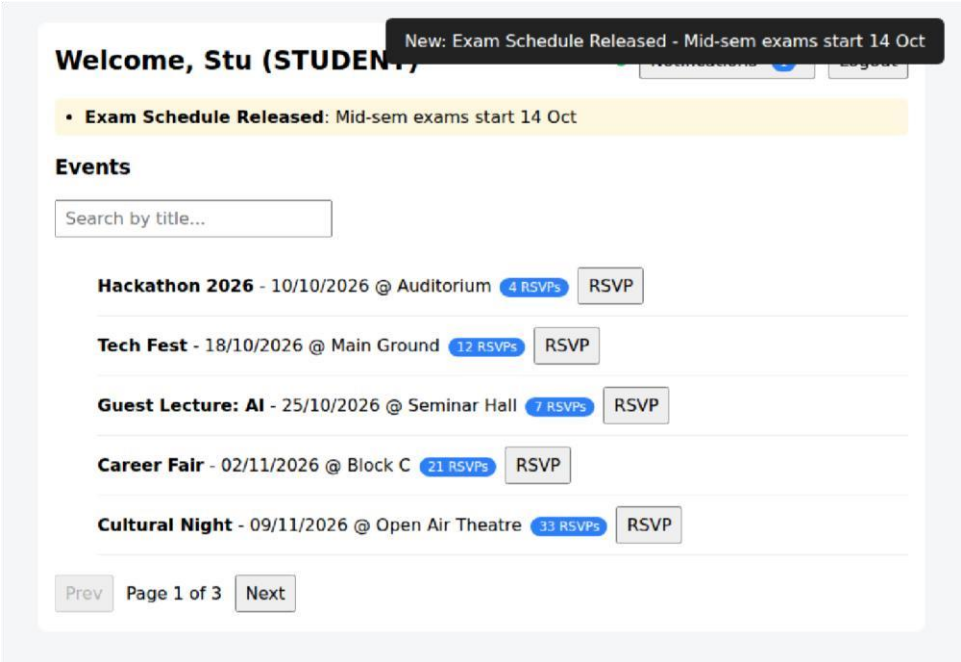
docker-compose.yml

```
version: "3.9"    services:   mongo:      image: mongo:7      container_name:
campusconnect-mongo     restart: unless-stopped     environment:
     MONGO_INITDB_ROOT_USERNAME: campusadmin
     MONGO_INITDB_ROOT_PASSWORD: campuspass       MONGO_INITDB_DATABASE: campusconnect
volumes:
-            mongo-data:/data/db     ports:
-            "27017:27017"     networks:       - campusnet      redis:      image: redis:7-alpine
container_name: campusconnect-redis     restart: unless-stopped     ports:
-            "6379:6379"     networks:       - campusnet      backend:
```

```
    build: ./backend
    container_name: campusconnect-backend
restart: unless-stopped     depends_on:       -
mongo       - redis     environment:       PORT:
5000
      MONGO_URI:
mongodb://campusadmin:campuspass@mongo:27017/campusconnect?authSource=admin       REDIS_HOST:
redis
      REDIS_PORT: 6379
      JWT_ACCESS_SECRET: change_me_access_secret
      JWT_REFRESH_SECRET: change_me_refresh_secret       FRONTEND_ORIGIN:
http://localhost:3000       NODE_ENV:
production     ports:       - "5000:5000"
networks:       - campusnet      frontend:     build:
./frontend
    container_name: campusconnect-frontend     restart: unless-stopped     depends_on:       - backend
environment:
      VITE_API_URL: http://localhost:5000/api       VITE_SOCKET_URL: http://localhost:5000
ports:       - "3000:80"     networks:       - campusnet     volumes:    mongo-data:     driver: local
networks:    campusnet:     driver: bridge
```




Welcome, Admin (ADMIN)
Logout
Admin Panel
Event title
mm/dd/yyyy
Location
Create Event
Announcement title
Message
Post Announcement
Events
Search by title...
Hackathon 2026 - 10/10/2026 @ Auditorium 4 RSVPs Delete
Tech Fest - 18/10/2026 @ Main Ground 12 RSVPs Delete
Guest Lecture: AI - 25/10/2026 @ Seminar Hall 7 RSVPs Delete
Career Fair - 02/11/2026 @ Block C 21 RSVPs Delete
Cultural Night - 09/11/2026 @ Open Air Theatre 33 RSVPs Delete


New: Exam Schedule Released - Mid-sem exams start 14 Oct
Welcome, Stu (STUDENT)
• Exam Schedule Released: Mid-sem exams start 14 Oct
Events
Search by title...
Hackathon 2026 - 10/10/2026 @ Auditorium 4 RSVPs RSVP
Tech Fest - 18/10/2026 @ Main Ground 12 RSVPs RSVP
Guest Lecture: AI - 25/10/2026 @ Seminar Hall 7 RSVPs RSVP
Career Fair - 02/11/2026 @ Block C 21 RSVPs RSVP
Cultural Night - 09/11/2026 @ Open Air Theatre 33 RSVPs RSVP
Prev
Page 1 of 3
Next